

## Shiv Nadar University Chennai

|                 |                                                           |
|-----------------|-----------------------------------------------------------|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science, Semester V |
| Subject         | CS3807 – Deep Learning Laboratory                         |
| Name            | R S Kirithic                                              |
| Section         | AIDS ‘B’                                                  |
| Register Number | 24011101082                                               |

### Experiment 2

#### Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

## 1. Objective

This experiment builds a Multi-Layer Perceptron in TensorFlow/Keras and applies it to the Fashion-MNIST dataset. This experiment goes through preprocessing, flattening, model construction, training, evaluation and then run an automated hyperparameter search to check whether a systematic search could match or improve on a manually chosen baseline architecture.

## 2. Tasks Performed

1. Explored the Fashion-MNIST dataset.
2. Preprocessed the images : normalized pixel values, flattened each image, and one hot encoded the labels.
3. Built a baseline MLP (Sequential, two ReLU hidden layers, Softmax output).
4. Trained the baseline model for 20 epochs and tracked its progress.
5. Evaluated the baseline model using standard classification metrics.
6. Ran an automated randomized hyperparameter search with 5-fold cross-validation.
7. Retrained the best configuration found by the search on the full training set.
8. Compared the baseline and optimized models on accuracy, precision, recall, F1-score, and training time.

## 3. Background Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer.

Each layer computes a weighted sum of its inputs and passes it through a non-linear activation called:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Fashion-MNIST is a classification problem, so the output layer uses a Softmax activation function so the ten outputs from the probability distribution gets mapped to the classes:

$$\hat{y} = \text{Softmax}(z)$$

The network is trained by minimizing the the loss function :categorical cross-entropy loss between the predicted and true class distributions:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

ReLU and Softmax are differentiable everywhere, which is what allows gradient-based backpropagation to train multi-layer networks The baseline (manually specified) architecture used in this experiment is:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

## 4. Dataset Description

Dataset: Fashion-MNIST

|                 |                                |
|-----------------|--------------------------------|
| Training Images | 60,000 – shape (60000, 28, 28) |
| Testing Images  | 10,000 – shape (10000, 28, 28) |
| Classes         | 10                             |
| Image Size      | $28 \times 28$                 |

Classes are : T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

## 5. Experimental Procedure and Results

### Task 1: Dataset Exploration

I loaded Fashion-MNIST via `keras.datasets.fashion_mnist` and printed the dataset dimensions, number of classes, and image size:

```
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape : (10000, 28, 28)
Testing labels shape : (10000,)
Number of classes    : 10
Image size           : 28 x 28
```

**Observation:** The images are  $28 \times 28$  grayscale, low resolution. So its almost impossible to see the textures of the clothing due it to be low resolution and grayscale ,so mostly the outline is what would help us distinguish different classes .

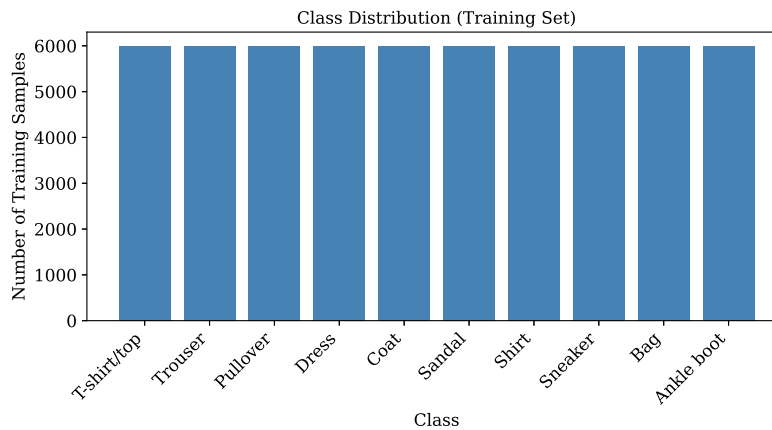


Figure 1: Class Distribution (Training Set).

**Observation:** There exactly 6,000 samples per class.The dataset is balanced, so we don't need to do class weight-ing or resampling was needed before training.

### Task 2: Data Preprocessing

Since Dense layers require a one-dimensional feature vector, each  $28 \times 28$  image needed to be flattened into a 784-dimensional vector ( $28 \times 28 = 784$ ). pipeline of steps:

1. Normalized pixel values from  $[0, 255]$  to  $[0, 1]$ .

2. Flattened each  $28 \times 28$  image into a 784-dimensional vector.
3. Converted integer labels into one-hot vectors (10 classes).

```
Shape before preprocessing:  
x_train: (60000, 28, 28) dtype: uint8  
x_test : (10000, 28, 28)
```

```
Shape after preprocessing:  
x_train: (60000, 784) dtype: float32  
x_test : (10000, 784)  
y_train: (60000, 10)  
y_test : (10000, 10)
```

```
Pixel value range: [0.00, 1.00]
```

### Task 3: Model Construction (Baseline)

MLP layer structure is : an input layer of 784 units, followed by two ReLU hidden layers (128 and 64 units), followed by a 10-unit Softmax output layer.

**Model: "Baseline\_MLP"**

| Layer (type)           | Output Shape | Param # |
|------------------------|--------------|---------|
| hidden_layer_1 (Dense) | (None, 128)  | 100,480 |
| hidden_layer_2 (Dense) | (None, 64)   | 8,256   |
| output_layer (Dense)   | (None, 10)   | 650     |

Total params: 109,386– all trainable.

### Task 4: Model Training (Baseline)

I compiled the baseline model with the Adam optimizer, categorical cross-entropy loss, and accuracy metric, then trained it for 20 epochs with batch size 32 and a 10% validation split.

| Epoch | Train Acc | Train Loss | Val Acc | Val Loss |
|-------|-----------|------------|---------|----------|
| 1     | 0.8234    | 0.4988     | 0.8510  | 0.3987   |
| 2     | 0.8646    | 0.3720     | 0.8593  | 0.3715   |
| 3     | 0.8774    | 0.3347     | 0.8618  | 0.3599   |
| 4     | 0.8857    | 0.3105     | 0.8647  | 0.3664   |
| 5     | 0.8917    | 0.2922     | 0.8673  | 0.3726   |
| 6     | 0.8978    | 0.2748     | 0.8675  | 0.3648   |
| 7     | 0.9026    | 0.2626     | 0.8703  | 0.3689   |
| 8     | 0.9064    | 0.2494     | 0.8678  | 0.3748   |
| 9     | 0.9096    | 0.2400     | 0.8697  | 0.3792   |
| 10    | 0.9135    | 0.2301     | 0.8687  | 0.3989   |
| 11    | 0.9174    | 0.2217     | 0.8767  | 0.3793   |
| 12    | 0.9202    | 0.2111     | 0.8707  | 0.3953   |
| 13    | 0.9234    | 0.2050     | 0.8758  | 0.3984   |
| 14    | 0.9251    | 0.1983     | 0.8755  | 0.4091   |
| 15    | 0.9272    | 0.1922     | 0.8712  | 0.4399   |
| 16    | 0.9307    | 0.1858     | 0.8768  | 0.4243   |
| 17    | 0.9311    | 0.1818     | 0.8770  | 0.4367   |
| 18    | 0.9329    | 0.1763     | 0.8793  | 0.4311   |
| 19    | 0.9358    | 0.1698     | 0.8787  | 0.4695   |
| 20    | 0.9382    | 0.1640     | 0.8835  | 0.4452   |

**training time on google collab: 166.43 seconds.**

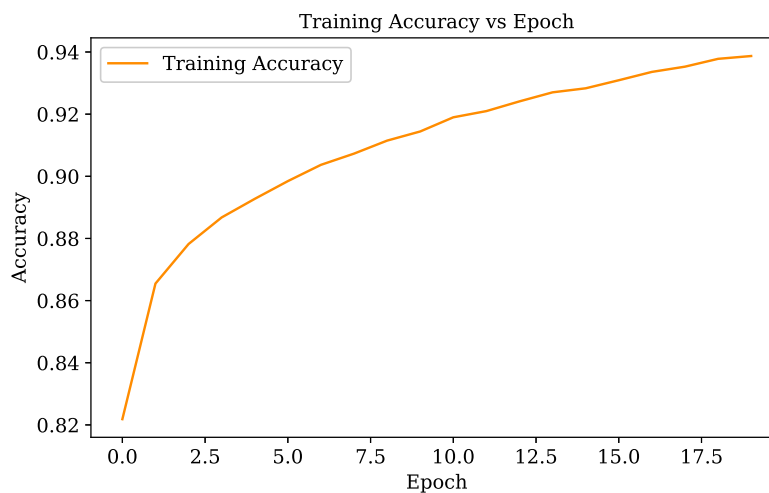


Figure 2: Training Accuracy vs. Epoch (Baseline).

**Observation:** Training accuracy goes from about 82% at epoch 1 to nearly 94% by epoch 20, flattening out toward the end. No spikes or drops in the curve.

**Observation:** Validation accuracy goes from 85.10% to 88.35% over the 20 epochs, following the training curve but with a gap that widens slightly over time. It slightly starts overfitting on the last few epochs(So even more epochs would make it overfit).

**Observation:** The Loss decreases every epoch, from about 0.50 to 0.16, with no bumps. Mirrors the accuracy curve above.

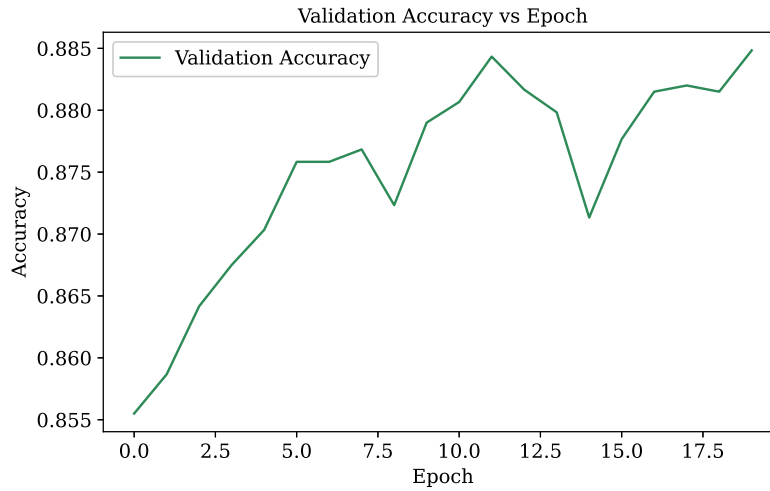


Figure 3: Validation Accuracy vs. Epoch (Baseline).

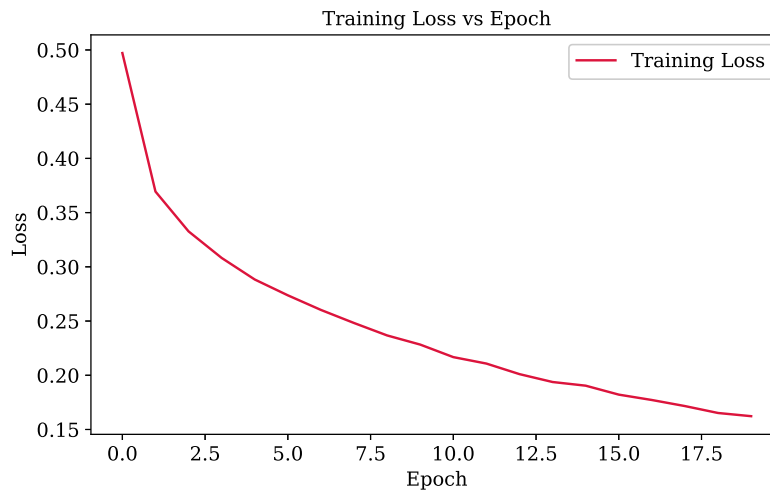


Figure 4: Training Loss vs. Epoch (Baseline).

**Observation:** Validation loss drops for the first few epochs, down to about 0.36 by epoch 3, then rises steadily to 0.4452 by epoch 20, even as training loss keeps falling. Overfitting starts around epoch 3–4.

### Task 5: Model Evaluation (Baseline)

I computed accuracy, weighted precision, weighted recall, and weighted F1-score on the 10,000-image test set.

Baseline Test Accuracy : 0.8753  
 Baseline Precision : 0.8765  
 Baseline Recall : 0.8753  
 Baseline F1-score : 0.8739

#### Classification Report (Baseline Model):

|             | precision | recall | f1-score | support |
|-------------|-----------|--------|----------|---------|
| T-shirt/top | 0.77      | 0.87   | 0.82     | 1000    |
| Trouser     | 0.99      | 0.97   | 0.98     | 1000    |
| Pullover    | 0.76      | 0.80   | 0.78     | 1000    |
| Dress       | 0.85      | 0.91   | 0.88     | 1000    |

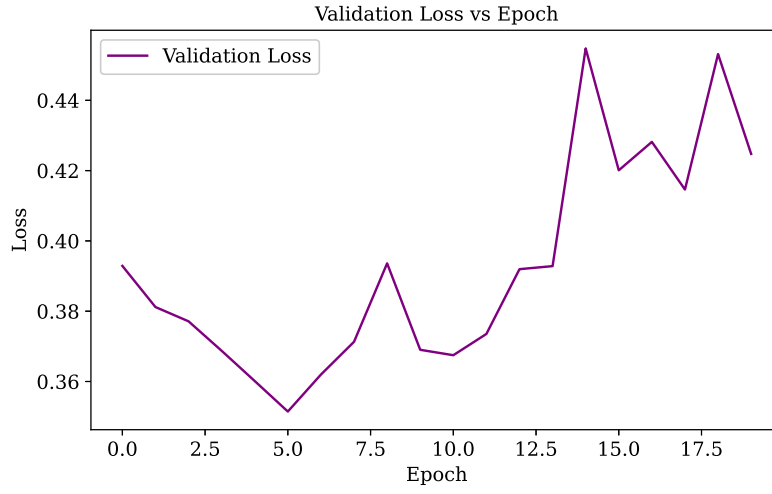


Figure 5: Validation Loss vs. Epoch (Baseline).

|              |      |      |      |       |
|--------------|------|------|------|-------|
| Coat         | 0.78 | 0.82 | 0.80 | 1000  |
| Sandal       | 0.96 | 0.97 | 0.97 | 1000  |
| Shirt        | 0.77 | 0.58 | 0.66 | 1000  |
| Sneaker      | 0.91 | 0.97 | 0.94 | 1000  |
| Bag          | 0.99 | 0.94 | 0.97 | 1000  |
| Ankle boot   | 0.98 | 0.92 | 0.95 | 1000  |
| accuracy     |      |      | 0.88 | 10000 |
| macro avg    | 0.88 | 0.88 | 0.87 | 10000 |
| weighted avg | 0.88 | 0.88 | 0.87 | 10000 |

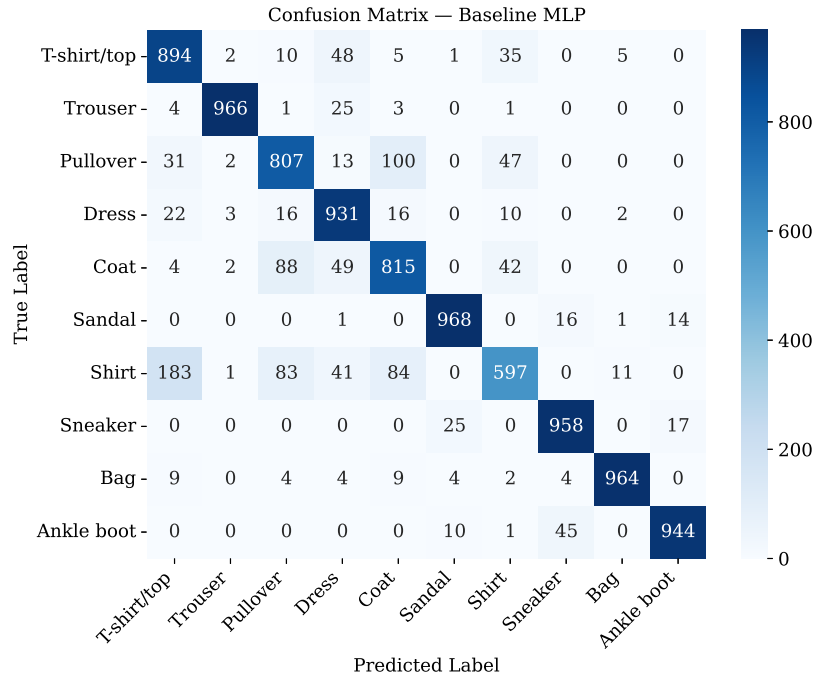


Figure 6: Confusion Matrix – Baseline MLP.

**Observation:** Most predictions fall on the diagonal. *Shirt* is the weakest class (recall 0.58), confused mainly with *T-shirt/top*, *Pullover*, and *Coat*. These are all upper-body garments with similar outlines at  $28 \times 28$  resolution, and

the model has no spatial information to distinguish them beyond raw pixel values.

## Task 6: Automated Hyperparameter Optimization

we can use a RandomizedSearchCV-style procedure + random sampling of hyperparameter combinations, each evaluated with 5-fold stratified cross-validation .

| Hyperparameter      | Candidate Values    |
|---------------------|---------------------|
| Hidden Layers       | 1, 2, 3             |
| Hidden Neurons      | 32, 64, 128, 256    |
| Learning Rate       | 0.1, 0.01, 0.001    |
| Batch Size          | 16, 32, 64, 128     |
| Epochs              | 10, 20, 30          |
| Optimizer           | SGD, Adam, RMSProp  |
| Activation Function | ReLU, Tanh, Sigmoid |
| Dropout Rate        | 0.0, 0.2, 0.5       |

The full search space contains  $3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3 = 11,664$  combinations, so I sampled 15 combinations (`n_iter` = 15) rather than searching exhaustively.

I defined a tunable model builder to construct an MLP with configurable hidden layers, hidden neurons, learning rate, optimizer, activation function, and dropout rate, then randomly sampled 15 combinations (`N_ITER` = 15) and evaluated each with 5-fold stratified cross-validation (`N_SPLITS` = 5) on a 6,000-image subsample of the training set, using a fixed reproducibility seed.

**Cross-validation search results (15 sampled combinations, 5-fold CV, subsampled on 6,000 training images):**

| #  | Optimizer | LR    | Neurons | Layers | Epochs | Dropout | Batch | Activation | Mean CV Acc ( $\pm$ std)               |
|----|-----------|-------|---------|--------|--------|---------|-------|------------|----------------------------------------|
| 1  | Adam      | 0.001 | 256     | 1      | 20     | 0.2     | 128   | Tanh       | <b>0.8422 (<math>\pm</math>0.0047)</b> |
| 2  | RMSProp   | 0.01  | 256     | 3      | 20     | 0.5     | 16    | ReLU       | 0.5838 ( $\pm$ 0.0168)                 |
| 3  | RMSProp   | 0.001 | 128     | 3      | 20     | 0.2     | 32    | Tanh       | 0.8310 ( $\pm$ 0.0137)                 |
| 4  | Adam      | 0.001 | 32      | 1      | 10     | 0.2     | 32    | Tanh       | 0.8360 ( $\pm$ 0.0127)                 |
| 5  | Adam      | 0.001 | 64      | 2      | 30     | 0.2     | 128   | Sigmoid    | 0.8287 ( $\pm$ 0.0078)                 |
| 6  | Adam      | 0.1   | 64      | 1      | 30     | 0.5     | 32    | Tanh       | 0.4677 ( $\pm$ 0.0642)                 |
| 7  | Adam      | 0.1   | 32      | 1      | 20     | 0.2     | 64    | Tanh       | 0.6302 ( $\pm$ 0.0355)                 |
| 8  | Adam      | 0.001 | 256     | 1      | 20     | 0.2     | 16    | ReLU       | 0.8337 ( $\pm$ 0.0105)                 |
| 9  | Adam      | 0.001 | 256     | 3      | 20     | 0.2     | 16    | Tanh       | 0.8262 ( $\pm$ 0.0164)                 |
| 10 | Adam      | 0.001 | 256     | 2      | 10     | 0.5     | 32    | Tanh       | 0.8282 ( $\pm$ 0.0094)                 |
| 11 | RMSProp   | 0.001 | 32      | 3      | 30     | 0.5     | 128   | Sigmoid    | 0.4967 ( $\pm$ 0.0227)                 |
| 12 | RMSProp   | 0.01  | 128     | 1      | 10     | 0.5     | 128   | Sigmoid    | 0.8022 ( $\pm$ 0.0108)                 |
| 13 | RMSProp   | 0.01  | 128     | 3      | 10     | 0.2     | 128   | Sigmoid    | 0.7777 ( $\pm$ 0.0332)                 |
| 14 | SGD       | 0.001 | 32      | 1      | 30     | 0.2     | 16    | Sigmoid    | 0.6817 ( $\pm$ 0.0111)                 |
| 15 | RMSProp   | 0.1   | 256     | 2      | 10     | 0.5     | 32    | ReLU       | 0.0995 ( $\pm$ 0.0023)                 |

**Best Hyperparameters Found** (combination #1): `hidden_layers` = 1, `hidden_neurons` = 256, `learning_rate` = 0.001, `optimizer` = adam, `activation` = tanh, `dropout_rate` = 0.2, `batch_size` = 128, `epochs` = 20. **Best cross-validation accuracy: 0.8422.**

**Observation:** Accuracy across the 15 trials ranges from 0.0995 (learning rate 0.1, RMSProp) to 0.8422 (best configuration). Most important : Learning rate and optimizer Less important : dropout, batch size, and number of layers

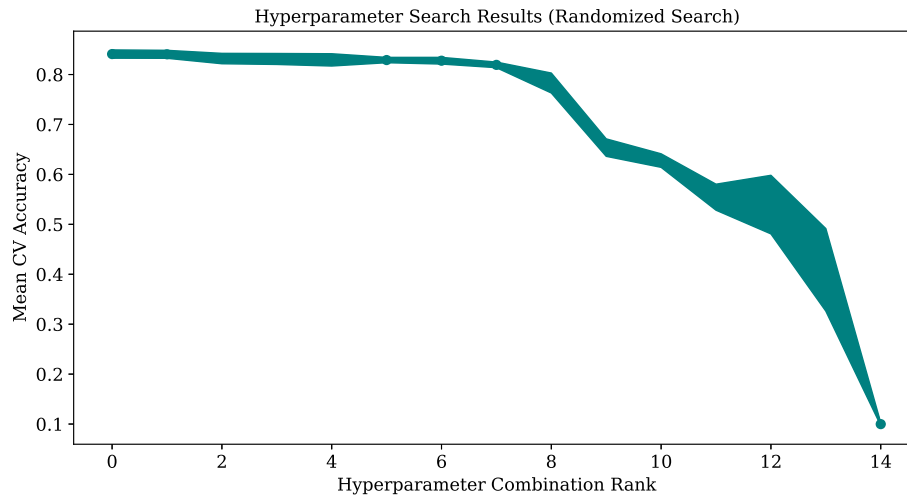


Figure 7: Hyperparameter Search Results (Randomized Search, 15 combinations, 5-fold CV).

### Task 7: Retraining with Optimized Hyperparameters (found in the above task)

rebuild the model using the tunable model builder and retrained it on the full training set for 20 epochs at batch size 128, with a 10% validation split.

**Model: "sequential\_1"** (optimized architecture)

| Layer (type)        | Output Shape | Param # |
|---------------------|--------------|---------|
| dense_3 (Dense)     | (None, 256)  | 200,960 |
| dropout_2 (Dropout) | (None, 256)  | 0       |
| dense_4 (Dense)     | (None, 10)   | 2,570   |

Total params: 203,530 (795.04 KB) – all trainable.



| Epoch | Train Acc | Train Loss | Val Acc | Val Loss |
|-------|-----------|------------|---------|----------|
| 1     | 0.8033    | 0.5513     | 0.8443  | 0.4290   |
| 2     | 0.8469    | 0.4300     | 0.8527  | 0.4031   |
| 3     | 0.8584    | 0.3951     | 0.8627  | 0.3712   |
| 4     | 0.8653    | 0.3743     | 0.8638  | 0.3671   |
| 5     | 0.8691    | 0.3591     | 0.8707  | 0.3554   |
| 6     | 0.8748    | 0.3444     | 0.8738  | 0.3406   |
| 7     | 0.8773    | 0.3342     | 0.8743  | 0.3394   |
| 8     | 0.8817    | 0.3239     | 0.8770  | 0.3358   |
| 9     | 0.8845    | 0.3153     | 0.8780  | 0.3371   |
| 10    | 0.8874    | 0.3067     | 0.8807  | 0.3332   |
| 11    | 0.8891    | 0.2990     | 0.8778  | 0.3296   |
| 12    | 0.8919    | 0.2929     | 0.8813  | 0.3196   |
| 13    | 0.8923    | 0.2884     | 0.8810  | 0.3226   |
| 14    | 0.8959    | 0.2812     | 0.8828  | 0.3183   |
| 15    | 0.8981    | 0.2746     | 0.8812  | 0.3252   |
| 16    | 0.8983    | 0.2727     | 0.8815  | 0.3230   |
| 17    | 0.9022    | 0.2655     | 0.8842  | 0.3155   |
| 18    | 0.9018    | 0.2636     | 0.8870  | 0.3104   |
| 19    | 0.9043    | 0.2579     | 0.8867  | 0.3104   |
| 20    | 0.9060    | 0.2531     | 0.8875  | 0.3110   |

**Optimized model training time: 88.70 seconds** (vs. 166.43 s for the baseline – a 46.7% reduction).

Optimized Test Accuracy : 0.8810  
 Optimized Precision : 0.8813  
 Optimized Recall : 0.8810  
 Optimized F1-score : 0.8798

### Classification Report (Optimized Model):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| T-shirt/top  | 0.80      | 0.87   | 0.83     | 1000    |
| Trouser      | 0.99      | 0.97   | 0.98     | 1000    |
| Pullover     | 0.76      | 0.84   | 0.80     | 1000    |
| Dress        | 0.85      | 0.91   | 0.88     | 1000    |
| Coat         | 0.84      | 0.77   | 0.80     | 1000    |
| Sandal       | 0.95      | 0.97   | 0.96     | 1000    |
| Shirt        | 0.75      | 0.62   | 0.68     | 1000    |
| Sneaker      | 0.93      | 0.95   | 0.94     | 1000    |
| Bag          | 0.98      | 0.97   | 0.97     | 1000    |
| Ankle boot   | 0.97      | 0.93   | 0.95     | 1000    |
| accuracy     |           |        | 0.88     | 10000   |
| macro avg    | 0.88      | 0.88   | 0.88     | 10000   |
| weighted avg | 0.88      | 0.88   | 0.88     | 10000   |

**Observation:** *Shirt* is still the weakest class, recall 0.62 (up from 0.58 in the baseline). *Coat* recall dropped slightly, to 0.77 from 0.82. Overall pattern is similar to the baseline’s confusion matrix, consistent with the close aggregate metrics between the two models.

### Task 8: Baseline vs. Optimized Comparison

I compared accuracy, precision, recall, F1-score, and training time between the baseline and optimized models.

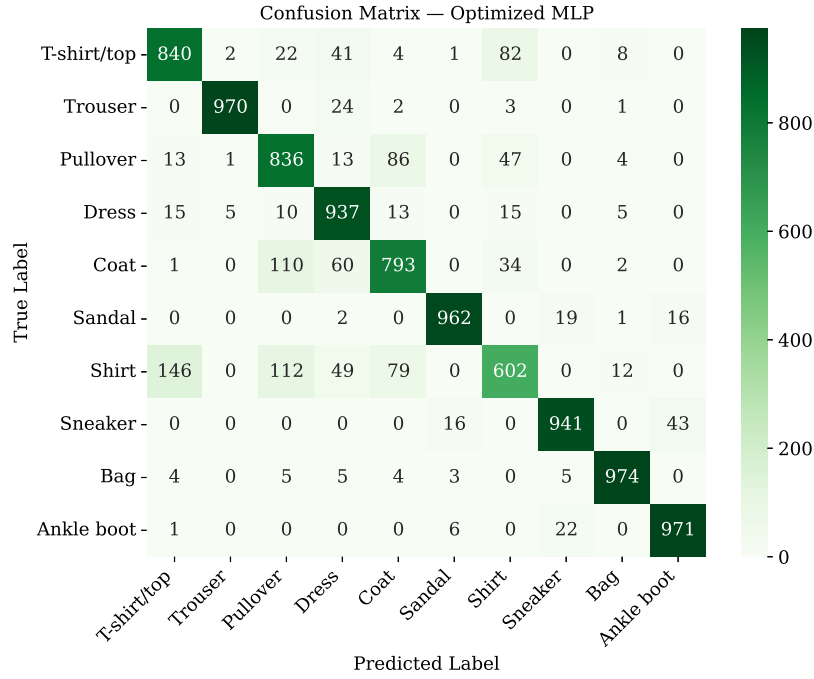


Figure 8: Confusion Matrix – Optimized MLP.

| Metric            | Baseline | Optimized |
|-------------------|----------|-----------|
| Accuracy          | 0.8753   | 0.8810    |
| Precision         | 0.8765   | 0.8813    |
| Recall            | 0.8753   | 0.8810    |
| F1-score          | 0.8739   | 0.8798    |
| Training Time (s) | 166.43   | 88.70     |

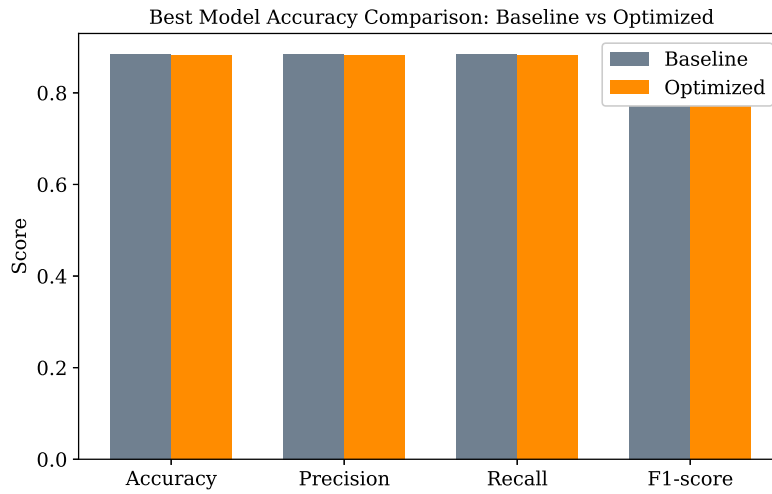


Figure 9: Best Model Accuracy Comparison: Baseline vs. Optimized.

**Observation:** The optimized model matches or exceeds the baseline on every metric and trains in roughly half the time (166.43 s vs. 88.70 s). A single wide hidden layer found via the search performed comparably to the

hand-picked two-layer baseline, with lower training time.

## 6. Performance Tables

### Best Hyperparameters

|                           |        |
|---------------------------|--------|
| Hidden Layers             | 1      |
| Hidden Neurons            | 256    |
| Learning Rate             | 0.001  |
| Batch Size                | 128    |
| Optimizer                 | Adam   |
| Activation Function       | Tanh   |
| Epochs                    | 20     |
| Dropout                   | 0.2    |
| Cross-validation Accuracy | 0.8422 |
| Testing Accuracy          | 0.8810 |

### Performance Comparison

| Metric            | Baseline | Optimized |
|-------------------|----------|-----------|
| Accuracy          | 0.8753   | 0.8810    |
| Precision         | 0.8765   | 0.8813    |
| Recall            | 0.8753   | 0.8810    |
| F1-score          | 0.8739   | 0.8798    |
| Training Time (s) | 166.43   | 88.70     |

## 7. Discussion

### 1. Which optimization method was used?

I used a RandomizedSearchCV-style procedure instead of a GridSearchCV this is because there are in total 11,664 combinations so i used randomized search with the parameter sampler with the 11,664-combination search space and each evaluated with 5-fold stratified cross-validation on a 6,000-image subsample.

### 2. Which hyperparameters were selected?

The final optimized hyperparameters are Hidden Layers = 1, Hidden Neurons = 256, Learning Rate = 0.001, Batch Size = 128, Optimizer = Adam, Activation = Tanh, Epochs = 20, Dropout = 0.2 which achieved a final validation score of 0.8422

### 3. Did optimization improve performance?

Yes , the test accuracy improved from 0.8753 to 0.8810 (optimized); precision from 0.8765 to 0.8813; recall from 0.8753 to 0.8810; F1-score from 0.8739 to 0.8798. It did improve but by that much the best improvement is that its a single layer so it trained much faster,

### 4. Which hyperparameter had the greatest impact?

Learning rate had by far the largest impact: every trial with learning rate 0.1 collapsed to poor accuracy (0.0995, 0.4677, 0.6302) .Learning rate of 0.001 is the best one .

### 5. Compare Grid Search and Randomized Search.

Grid Search evaluates every combination in the search space (here, 11,664 combinations), which guarantees

the optimum within the grid but scales combinatorially with the cross-validation at this scale. Randomized Search samples a fixed, much smaller number of combinations (15 here); it trades the guarantee of finding the true optimum for far better computational efficiency, and, as I saw, it still located a strong configuration (0.8422 CV accuracy) in a small fraction of the compute an exhaustive search would have needed.

#### 6. Recommend the best MLP configuration.

I would recommend the configuration found by the search: a single hidden layer of 256 units with Tanh activation and 0.2 dropout, trained with the Adam optimizer (learning rate 0.001) for 20 epochs at batch size 128. It achieved the best cross-validation accuracy (0.8422) among all 15 sampled configurations from the randomized Search, generalized to 0.8810 test accuracy, and trained in roughly half the time of the deeper, manually chosen baseline (88.70 s vs. 166.43 s).

- Learning rate had the largest effect on hyperparameter search results. Every bad combination had the learning rate 0.1, regardless of other settings.
- Both the baseline and the optimized model performed worst on the *Shirt* class, suggesting the bottleneck is the input data itself representation losing spatial information, rather than the specific architecture used.

## 8. Conclusion

- I Implemented an MLP classifier for Fashion-MNIST and I have done the complete pipeline from data preprocessing (normalization, flattening to 784 dimensions, one-hot encoding) through model construction, training, evaluation, and automated hyperparameter optimization.
- The manually specified baseline (784  $\rightarrow$  128  $\rightarrow$  64  $\rightarrow$  10, Adam, 20 epochs, batch 32) reached **87.53% test accuracy** in 166.43 s.
- A 15-combination randomized search with 5-fold cross-validation identified a shallower-but-wider configuration (784  $\rightarrow$  256 (Tanh, dropout 0.2)  $\rightarrow$  10, Adam, learning rate 0.001, batch 128, 20 epochs) that reached **88.10% test accuracy** in 88.70 s on the full training set – a small accuracy gain alongside a substantial (46.7%) reduction in training time.
- Both models struggled most with the *Shirt* class, which is visually similar to *T-shirt/top*, *Pullover*, and *Coat*, indicating that flattened-pixel MLPs discard the spatial structure that would help disambiguate these garments – a limitation that convolutional architectures are better suited to address and also the data is too simplified .
- Overall, this experiment demonstrated the practical benefit of systematic, automated hyperparameter search over manual tuning, both in final accuracy and in training efficiency, and built directly on the foundations (weights, biases, activation functions, gradient-based learning) laid down in Experiment 1.

## 9. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

## Additional Task

### Perceptron Learning Algorithm for AND, OR, and NOT Gates (K4)

## Objective

A Single Layer Perceptron was coded from scratch in Python to learn the AND, OR, and NOT logic gates. For each gate, the weights and bias were printed after every single weight update, and the decision boundary was plotted after every update.

## Training Log

### AND gate:

```
Epoch 1, sample [0 0], target 0, pred 1 -> weights=[0. 0.], bias=-0.10
Epoch 1, sample [1 1], target 1, pred 0 -> weights=[0.1 0.1], bias=0.00
Epoch 2, sample [0 0], target 0, pred 1 -> weights=[0.1 0.1], bias=-0.10
Epoch 2, sample [0 1], target 0, pred 1 -> weights=[0.1 0. ], bias=-0.20
Epoch 2, sample [1 1], target 1, pred 0 -> weights=[0.2 0.1], bias=-0.10
Epoch 3, sample [0 1], target 0, pred 1 -> weights=[0.2 0. ], bias=-0.20
Epoch 3, sample [1 0], target 0, pred 1 -> weights=[0.1 0. ], bias=-0.30
Epoch 3, sample [1 1], target 1, pred 0 -> weights=[0.2 0.1], bias=-0.20
Converged after epoch 4, no errors left.
```

### OR gate:

```
Epoch 1, sample [0 0], target 0, pred 1 -> weights=[0. 0.], bias=-0.10
Epoch 1, sample [0 1], target 1, pred 0 -> weights=[0. 0.1], bias=0.00
Epoch 2, sample [0 0], target 0, pred 1 -> weights=[0. 0.1], bias=-0.10
Epoch 2, sample [1 0], target 1, pred 0 -> weights=[0.1 0.1], bias=0.00
Epoch 3, sample [0 0], target 0, pred 1 -> weights=[0.1 0.1], bias=-0.10
Converged after epoch 4, no errors left.
```

### NOT gate:

```
Epoch 1, sample [1], target 0, pred 1 -> weights=[-0.1], bias=-0.10
Epoch 2, sample [0], target 1, pred 0 -> weights=[-0.1], bias=0.00
Converged after epoch 3, no errors left.
```

## Decision Boundary Plots

**Observation:** AND converges in 8 updates, OR in 5 updates, and NOT in 2 updates. All three gates are linearly separable, so the perceptron finds a boundary that classifies every sample correctly, unlike the XOR case, which never converges.

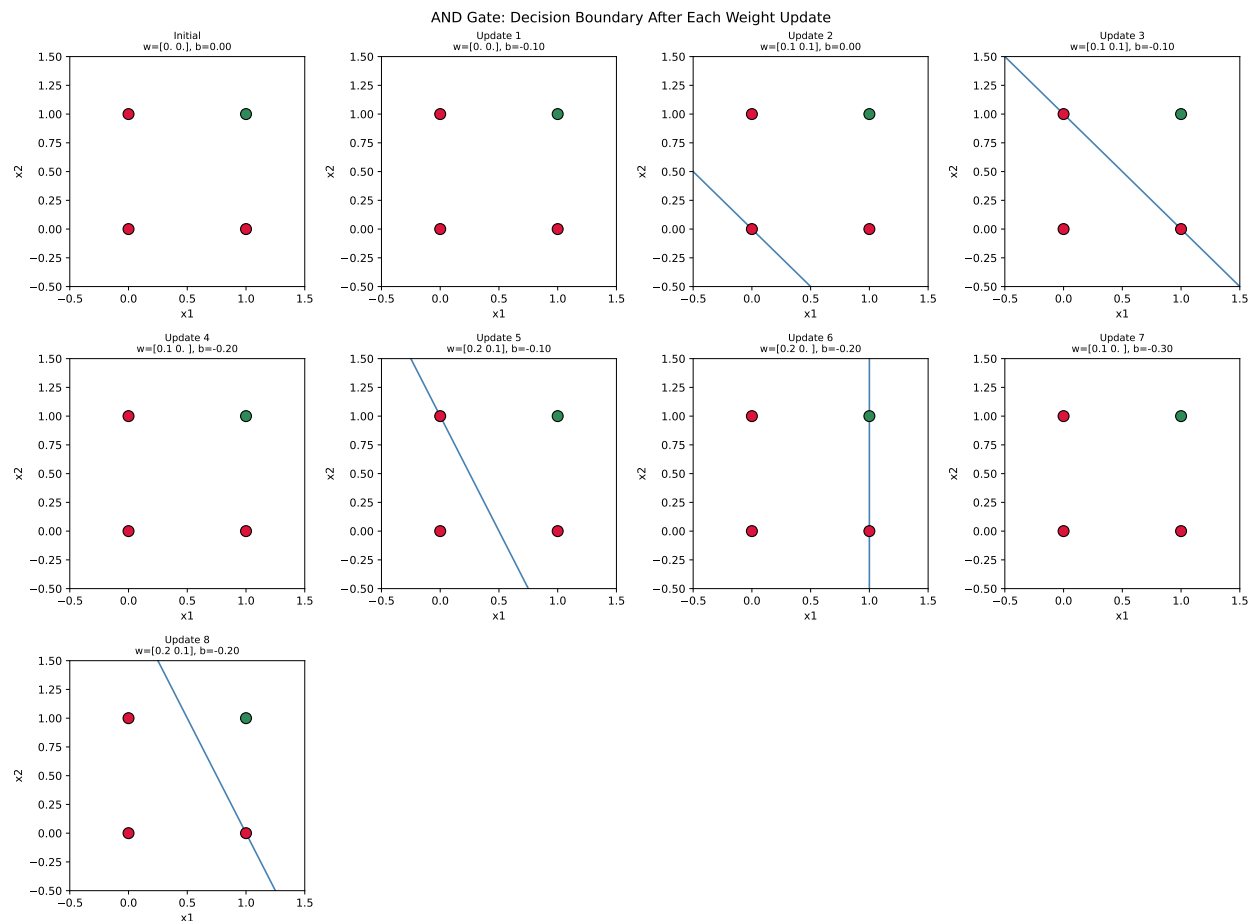


Figure 1: AND Gate: Decision Boundary After Each Weight Update.

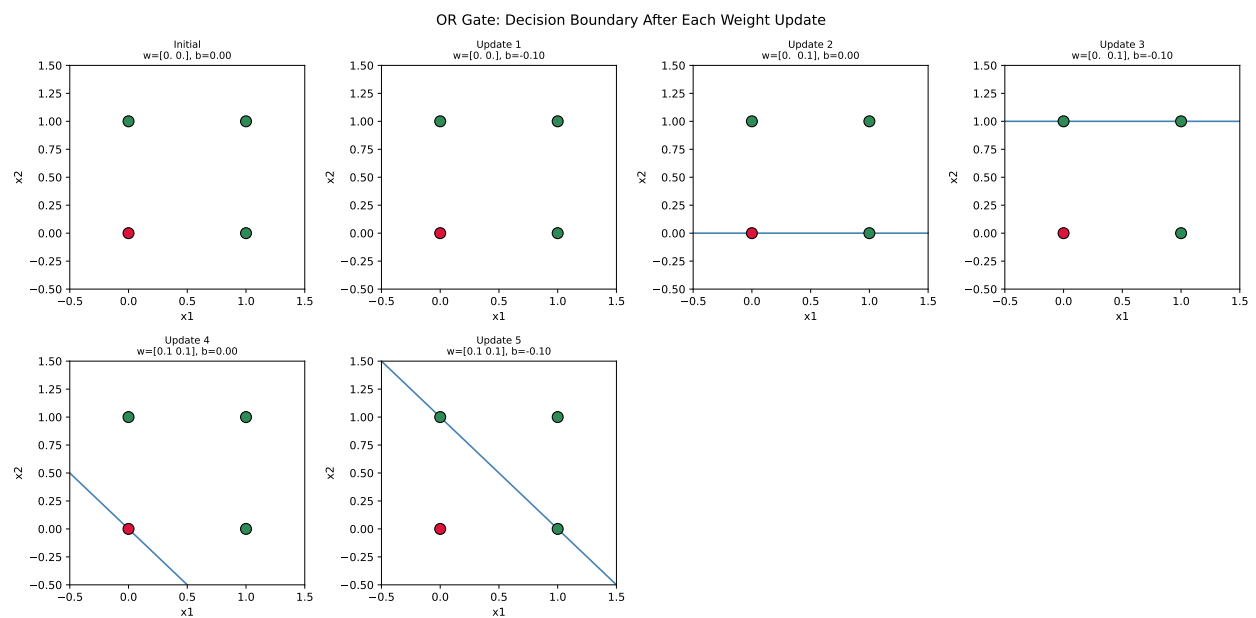


Figure 2: OR Gate: Decision Boundary After Each Weight Update.

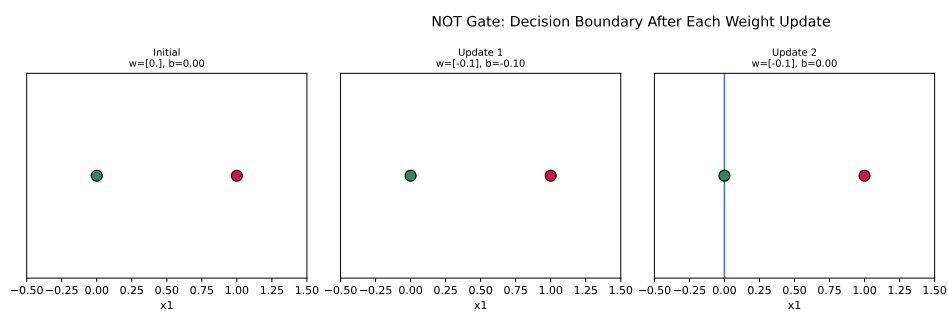


Figure 3: NOT Gate: Decision Boundary After Each Weight Update.